

SQL & PostgreSQL

Talking to databases — store data and pull out exactly what you need

AI/ML Engineer Learning Series · Deep Dive Edition

SQL	A language to ask questions to a database
PostgreSQL	A popular free database that understands SQL
Why learn	Most real-world data lives in databases, not CSV files
For ML	Pull and prepare data before loading into Pandas
Good news	SQL reads almost like plain English

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is a Database?	Tables, rows, columns — and SQL
2	SELECT	Pull data out of a table
3	WHERE	Get only the rows you want
4	ORDER BY & LIMIT	Sort and take the top few
5	Aggregate Functions	COUNT, SUM, AVG, MIN, MAX
6	GROUP BY	Group rows and summarise each group
7	JOIN	Combine two tables together
8	INSERT, UPDATE, DELETE	Change the data
9	CREATE TABLE	Make a new table
10	PostgreSQL Extras	What makes Postgres special
11	SQL + Python	Use SQL inside your ML work
12	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference of every command


```
-- Get all columns from the students table
SELECT * FROM students;      -- the * means 'every column'

-- Get only the name and score columns
SELECT name, score FROM students;

-- Rename a column in the result (alias)
SELECT name AS student_name, score AS marks FROM students;

-- Get only unique cities (no repeats)
SELECT DISTINCT city FROM students;
```

■ The semicolon ; at the end tells the database 'this query is finished'. Always add it.

3. WHERE — Filter the Rows

WHERE lets you pick only the rows that match a condition (■■■■). This is just like filtering in Pandas, but in plain words.

```
-- Students with score above 80
SELECT * FROM students WHERE score > 80;

-- Students from Sylhet
SELECT * FROM students WHERE city = 'Sylhet';

-- Combine conditions with AND / OR
SELECT * FROM students WHERE city = 'Sylhet' AND score > 90;
SELECT * FROM students WHERE city = 'Sylhet' OR city = 'Dhaka';

-- A range of values
SELECT * FROM students WHERE score BETWEEN 70 AND 90;

-- Match a list of values
SELECT * FROM students WHERE city IN ('Sylhet','Dhaka','Ctg');

-- Match a text pattern (LIKE). % means 'any letters'
SELECT * FROM students WHERE name LIKE 'S%';  -- names starting with S
SELECT * FROM students WHERE name LIKE '%a%';  -- names containing 'a'
```

Operator	Meaning	Example
=	equal to	city = 'Dhaka'
!= or <>	not equal to	city != 'Dhaka'
> , <	greater / less than	score > 80
>= , <=	greater-or-equal / less-or-equal	score >= 50
BETWEEN	inside a range	score BETWEEN 70 AND 90

IN	matches any in a list	city IN ('A','B')
LIKE	matches a text pattern	name LIKE 'S%'
IS NULL	value is empty	score IS NULL

4. ORDER BY & LIMIT — Sort and Trim

ORDER BY sorts the result. **LIMIT** keeps only the first few rows. Together they answer questions like 'who are the top 3 students?'

```
-- Sort by score, lowest first (default = ascending)
SELECT * FROM students ORDER BY score;

-- Sort by score, highest first
SELECT * FROM students ORDER BY score DESC;    -- DESC = descending

-- Sort by city, then by score within each city
SELECT * FROM students ORDER BY city ASC, score DESC;

-- Top 3 students by score
SELECT * FROM students ORDER BY score DESC LIMIT 3;

-- Skip the first 3, then take 5 (for paging)
SELECT * FROM students ORDER BY score DESC LIMIT 5 OFFSET 3;
```

5. Aggregate Functions — Summarise

Aggregate (COUNT, SUM, AVG, MIN, MAX) functions take many rows and give back one summary number. Like 'what is the average score of everyone?'

Function	What it gives
COUNT(*)	How many rows
SUM(score)	Total of all scores
AVG(score)	Average score
MIN(score)	Smallest score
MAX(score)	Biggest score

```
SELECT COUNT(*) FROM students;           -- total students
SELECT AVG(score) FROM students;         -- average score
SELECT MAX(score), MIN(score) FROM students;

-- Count students from Sylhet only
SELECT COUNT(*) FROM students WHERE city = 'Sylhet';
```

6. GROUP BY — Summarise Per Group

GROUP BY is the powerful one. It splits rows into groups, then runs an aggregate on each group. Example: 'average score for EACH city' (not everyone together). This is the exact same idea as groupby in Pandas.

```
-- Average score for each city
SELECT city, AVG(score)
FROM students
GROUP BY city;

-- How many students in each city
SELECT city, COUNT(*) AS num_students
FROM students
GROUP BY city;

-- Only show groups where average score is above 85
-- (HAVING filters groups, like WHERE filters rows)
SELECT city, AVG(score) AS avg_score
FROM students
GROUP BY city
HAVING AVG(score) > 85
ORDER BY avg_score DESC;
```

■ **WHERE vs HAVING** — *WHERE filters rows BEFORE grouping. HAVING filters groups AFTER grouping. Use HAVING when your condition uses COUNT/AVG/SUM.*

7. JOIN — Combine Two Tables

Data is often split across many tables. One table for students, another for their scores. **JOIN** sticks them together using a shared column (usually an id). This keeps data tidy and is a core database idea.

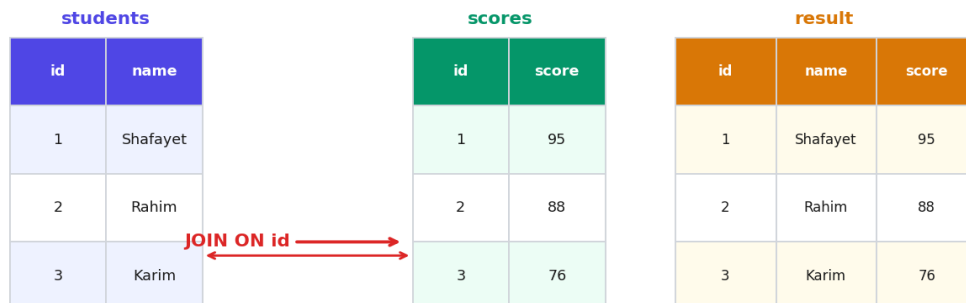


Fig 2 — JOIN matches rows from two tables using the shared 'id' column.

```
SELECT students.name, scores.score
FROM students
JOIN scores ON students.id = scores.id;

-- Short version using table nicknames (aliases)
SELECT s.name, sc.score
FROM students AS s
JOIN scores AS sc ON s.id = sc.id;
```

Join Type	What it keeps
INNER JOIN	Only rows that match in BOTH tables (most common)
LEFT JOIN	All rows from the left table + matches from right
RIGHT JOIN	All rows from the right table + matches from left
FULL JOIN	All rows from both tables

```
-- Keep ALL students, even those with no score yet
SELECT s.name, sc.score
FROM students AS s
LEFT JOIN scores AS sc ON s.id = sc.id;
```

8. INSERT, UPDATE, DELETE — Change Data

So far we only read data. These three commands change it: add new rows, change existing rows, or remove rows.

```
-- ADD a new row
INSERT INTO students (id, name, city, score)
VALUES (5, 'Mitu', 'Barisal', 84);

-- CHANGE an existing row (ALWAYS use WHERE!)
UPDATE students
SET score = 90
WHERE id = 5;

-- REMOVE a row (ALWAYS use WHERE!)
DELETE FROM students WHERE id = 5;
```

■ **DANGER:** *UPDATE or DELETE without a WHERE will change or delete EVERY row in the table. Always double-check your WHERE before running these.*

9. CREATE TABLE — Make a New Table

Before storing data you make a table and tell it what columns it has and what type each column is (number, text, date).

```
CREATE TABLE students (
  id      SERIAL PRIMARY KEY,  -- auto-numbered unique id
  name    VARCHAR(100) NOT NULL, -- text, cannot be empty
  city    VARCHAR(50),         -- text
  score   INTEGER,             -- whole number
  joined  DATE DEFAULT CURRENT_DATE -- date, default = today
);
```

Data Type	Stores	Example
INTEGER	Whole numbers	42, -7
DECIMAL / NUMERIC	Numbers with decimals	3.14, 99.50
VARCHAR(n)	Text up to n characters	'Sylhet'
TEXT	Long text, any length	a paragraph
BOOLEAN	True or False	TRUE
DATE	A calendar date	2026-06-23
TIMESTAMP	Date + time	2026-06-23 14:30
SERIAL	Auto-counting id	1, 2, 3...

10. What Makes PostgreSQL Special

SQL is the language; PostgreSQL is one program that speaks it. Almost all SQL is the same everywhere, but Postgres has some nice extra powers:

- **Free and open-source** — no cost, used by huge companies.
- **JSON support** — can store flexible JSON data inside a column.
- **Window functions** — advanced calculations like running totals and rankings.
- **Full-text search** — search inside text like a mini search engine.
- **Strong and reliable** — handles huge data without breaking.

A Taste of Window Functions

A window function (■■■■■■ ■■■■■■) does math across rows while keeping every row visible. Example: rank students by score without grouping them away.

```
SELECT name, score,
       RANK() OVER (ORDER BY score DESC) AS rank
FROM students;
-- Each student keeps their row, plus gets a rank number
```

11. Using SQL Inside Python

In ML work, you often pull data from a database straight into a Pandas table. Pandas can run a SQL query for you and hand back a DataFrame ready to use.

```
import pandas as pd
from sqlalchemy import create_engine

# Connect to a PostgreSQL database
engine = create_engine(
    'postgresql://user:password@localhost:5432/mydb')

# Write a SQL query and load the result into a DataFrame
query = '''
    SELECT city, AVG(score) AS avg_score
    FROM students
    GROUP BY city
    ORDER BY avg_score DESC;
'''

df = pd.read_sql(query, engine)

print(df.head())  # now it's a normal Pandas table!
```

■ *This is the real workflow: SQL gets and trims the data on the database side (fast, even for millions of rows), then Pandas takes over for the ML steps.*

12. Common Mistakes

Mistake	What happens	Fix
UPDATE/DELETE with no WHERE	Changes every single row	Always add WHERE first
Forgetting the semicolon ;	Query won't run	End every query with ;
= for text without quotes	Error	Use quotes: city = 'Dhaka'
SELECT col not in GROUP BY	Error in Postgres	Group by every plain column
Using WHERE for group filter	Error with COUNT/AVG	Use HAVING for group conditions
Joining on the wrong column	Wrong or doubled rows	Join on the shared id

Quick Reference Cheatsheet

Task	SQL
Get all columns	SELECT * FROM table;
Get some columns	SELECT a, b FROM table;
Unique values	SELECT DISTINCT col FROM table;
Filter rows	SELECT * FROM table WHERE col > 5;
AND / OR	WHERE a = 1 AND b = 2;
Range	WHERE col BETWEEN 1 AND 10;
In a list	WHERE col IN ('a','b');
Text pattern	WHERE name LIKE 'S%';
Sort	ORDER BY col DESC;
Top N rows	ORDER BY col DESC LIMIT 5;
Count rows	SELECT COUNT(*) FROM table;
Average	SELECT AVG(col) FROM table;
Group summary	SELECT c, AVG(x) FROM t GROUP BY c;
Filter groups	GROUP BY c HAVING COUNT(*) > 3;
Join tables	FROM a JOIN b ON a.id = b.id;
Left join	FROM a LEFT JOIN b ON a.id = b.id;
Add a row	INSERT INTO t (..) VALUES (..);
Change a row	UPDATE t SET col = 1 WHERE id = 5;
Delete a row	DELETE FROM t WHERE id = 5;
Make a table	CREATE TABLE t (id SERIAL PRIMARY KEY);
Load into Pandas	pd.read_sql(query, engine)

Next Topic: EDA (Exploratory Data Analysis) — putting Pandas, Matplotlib, and Seaborn together to truly understand a dataset before building any model.